

"Sir/Ma'am, our project is on Gender and Age Detection Using Deep Learning."

The idea is simple — given a face image or live webcam feed, our system automatically detects the person's gender (Male/Female) and approximate age group in real-time.

How it works — step by step:

First, the input image is passed through a Face Detector — we used an SSD model with ResNet-10 backbone, loaded via OpenCV's DNN module. It draws a bounding box around every face it finds.

Then for each detected face, we extract that region and pass it through two separate pre-trained CNN models — both are Caffe-based models trained on the Adience dataset:

The Gender model gives a probability for Male or Female

The Age model gives probabilities across 8 age groups — like (0-2), (4-6), (8-12), (25-32), up to (60-100)

The label with the highest probability is selected and displayed on the image as an overlay — like "Female, (25-32)"

Key point — we did not train any model from scratch. We used pre-trained weights, which saved time and computational resources. This is called transfer learning / inference.

The entire project is built in Python using Jupyter Notebook with OpenCV, NumPy, and Matplotlib.

Results: Gender accuracy was above 90% under normal lighting. Webcam ran at 25-30 FPS on a standard laptop without GPU.

Limitation: Age is predicted as a range, not exact value. Accuracy drops in poor lighting or side-angle faces.

"Sir/Ma'am, our project is on Parallel Quicksort using MPI."

Quicksort is a fast sorting algorithm with $O(n \log n)$ average complexity, but for large datasets, sequential execution becomes slow. To solve this, we used MPI — Message Passing Interface — to distribute the work across multiple processes running simultaneously.

How it works:

The master process (rank 0) takes input from the user

MPI_Scatter is used to distribute the data — it splits the array into equal chunks and sends one chunk to each process. For example, if we have 8 elements and 4 processes, each process gets 2 elements.

Each process then independently sorts its own chunk using Quicksort — this is where parallel execution happens

MPI_Gather is used to collect results — it brings back all the sorted chunks from every process back to the master

Finally, the master performs one last Quicksort on the full gathered data and displays the sorted output

So in simple terms — Scatter = distribute, Gather = collect.

Key benefits are reduced execution time, better CPU utilization, and scalability.

Main challenges are communication overhead and load balancing. If the number of elements is not divisible by number of processes, the program throws an error and aborts.

We used C++ with MS-MPI on Windows, compiled with G++, and ran using `mpiexec -n 4`.

Input 8 3 7 4 9 2 6 5 → Output 2 3 4 5 6 7 8 9 